



دانشگاه شاهد

Shahed University

آنالیز شبکه های اجتماعی

Social Network Analysis

استاد راهنما :

خانم دکتر پوربهمن

توسعه دهندگان :

امیدرئیزی

امیرمهدی زارعان

امیرحسین شورابی

علی فلاح

در دنیای امروز، شبکه‌های اجتماعی نقش بسیار مهمی در ارتباطات ایفا می‌کنند و بررسی ساختار این ارتباطات برای سازمان‌ها و شرکت‌ها ارزش بالایی دارد. هدف از اجرای این پروژه، طراحی و توسعه یک ابزار نرم‌افزاری مستقل برای استخراج، تحلیل و پردازش اطلاعات شبکه‌های اجتماعی است. در این سیستم، شبکه اجتماعی به شکل یک تئوری گراف مدل‌سازی شده است؛ به این صورت که کاربران به عنوان راس‌های گراف و روابط دوستی میان آن‌ها به عنوان یال‌های گراف در نظر گرفته شده‌اند.

هسته اصلی این نرم‌افزار وظیفه دارد پردازش‌های زیر را با کمترین پیچیدگی زمانی ممکن انجام دهد

نمایش دوستان : لیست کردن تمام دوستان یک کاربر مشخص .

بررسی ارتباط : تشخیص اینکه آیا دو کاربر در شبکه با یکدیگر ارتباط (دوستی) دارند یا خیر .

مسیریابی : پیدا کردن کوتاه‌ترین مسیر ممکن بین دو کاربر .

سیستم پیشنهاد دوست : ارائه پیشنهاد‌های دوستی به کاربران بر اساس ساختار گراف .

تشخیص گروه‌ها : شناسایی و لیست کردن گروه‌های مختلف شبکه (مولفه‌های همبند) و اعضای آن‌ها .

کاربران محبوب : یافتن کاربری که بیشترین تعداد دوست را در سطح شبکه دارد .

دوستان مشترک : پیدا کردن دوستان مشترک میان دو کاربر .

آمار و اطلاعات کلان شبکه : محاسبه تعداد کل کاربران، تعداد کل دوستی‌ها، میانگین روابط به ازای هر کاربر، یافتن بزرگترین گروه دوستی ، شناسایی کاربر با بیشترین دوست ، قطر گراف و چگالی گراف .

محاسبه فواصل : محاسبه فاصله یک کاربر خاص از تمامی افراد شبکه و مرتب‌سازی آن‌ها از کمترین تا بیشترین فاصله .

یافتن افراد کلیدی (پل‌های ارتباطی) : شناسایی کاربرانی که بیشترین مرکزیت بینایی را دارند؛ به این معنا که بیشترین حضور را در کوتاه‌ترین مسیرهای شبکه دارند و حذف آن‌ها باعث از هم پاشیدن شبکه می‌شود.

تشخیص جوامع دوستی پیشرفته : توانایی تفکیک اکیپ‌ها و جوامع فشرده در دل یک گروه بزرگتر (زیرگراف‌هایی با تراکم یال بالا درونی و یال‌های خروجی اندک).

بیشینه‌سازی انتشار خبر : شناسایی کاربر بهینه در شبکه که در صورت دریافت یک خبر، می‌تواند آن را در کمترین زمان ممکن به بیشترین تعداد افراد شبکه برساند .

نکات کلی درباره توسعه این سیستم

ذخیره‌سازی داده‌ها : اطلاعات مربوط به شبکه اجتماعی در قالب فایل‌های JSON ذخیره و بازیابی می‌شوند.

مدیریت داده‌ها : سیستم قابلیت افزودن، ویرایش و حذف کاربران را به صورت بهینه دارا می‌باشد.

اولویت توسعه : هدف اصلی در تمام بخش‌های سیستم، استفاده از داده‌ساختارها و الگوریتم‌های بهینه برای مینیمم کردن پیچیدگی زمانی پردازش‌ها بوده است.

معماری نرم‌افزار

در پروژه‌های تحلیل شبکه، الگوریتم‌های پردازش گراف بسیار سنگین و پیچیده هستند. اگر منطق این الگوریتم‌ها با فرمت‌های نمایش در رابط کاربری (UI) یا منطق تبدیل داده‌ها (مانند تبدیل آیدی به نام کاربر) ترکیب شود، کد به شدت شکننده شده و با کوچک‌ترین تغییر در UI یا نحوه دریافت ورودی، الگوریتم‌های گراف دچار خطا می‌شوند.

به همین دلیل، در این پروژه از الگوی **معماری تمیز (clean Architecture)** استفاده شده است. فلسفه اصلی این معماری جداسازی کامل دغدغه‌ها است.

1- لایه Core (هسته پردازشی و الگوریتم‌ها) :

این لایه از 4 بخش مختلف تشکیل شده است ، در بخش اول یعنی Model گراف ما ذخیره شده است برای دسترسی راحت تر و اجرای یک سری متد های خاص . در بخش دوم یعنی Results خروجی تمامی الگوریتم ها در قالب یک کلاس طراحی شده است برای توسعه بهتر . در بخش سوم این لایه که Algorithms هست پیاده سازی تمامی الگوریتم های مورد نیاز در پروژه انجام شده که اینترفیس های این الگوریتم ها در بخش چهارم یعنی Abstractions قرار دارد.

2 - لایه Application :

در بخش اول این لایه (services) تمامی سرویس های مورد نیاز برای پروژه به دو قسمت دسته بندی شده اند ، یکی Queries که مربوط به سرویس هایی هستند که فقط اطلاعات را از گراف میخوانند و هیچ تغییری در گراف نمیدهند و یکی Commands که مربوط به سرویس هایی هست که گراف را تغییر میدهند مثل اضافه کردن کاربر یا دوستی . بخش دوم DTO ها هستند که کلاس هایی هستند که کنترل میکنند خروجی هر الگوریتم متناسب با نیاز لایه UI به دست این لایه برسد . بخش سوم Abstractions هست که اینترفیس تمامی سرویس های ما در این بخش قرار دارد . بخش چهارم یعنی contracts اینترفیس کلاس هایی هستند که پیاده سازی آن در لایه دیگری انجام شده اما در این لایه استفاده میشود.

3 - لایه (UI) Presentation :

این لایه فقط و فقط وظیفه گرفتن ورودی از کاربر و رندر کردن DTO های دریافتی از لایه Application را دارد. هیچ گونه الگوریتم گراف یا محاسبه ریاضی در این لایه وجود ندارد.

4 - لایه Infrastructure :

بخش اول این لایه به نام Storage نحوه کار با فایل های json را مشخص میکند . بخش دوم یعنی Persistence اطلاعاتی که از فایل های json گرفته شده را مدیریت میکند و بخش سوم یعنی Runtime قوانین مربوط به Persistence را تعیین میکند و اطلاعات را در RAM نگهداری میکند .

تشریح الگوریتم‌ها و تحلیل پیچیدگی زمانی و فضایی

در این فصل، هسته مرکزی پردازش‌های سیستم و منطق ریاضی پشت هر یک از نیازمندی‌های شبکه اجتماعی مورد بررسی قرار می‌گیرد. با توجه به اهمیت سرعت اجرا و مصرف بهینه منابع حافظه در گراف‌های مقیاس بزرگ، تمامی عملیات‌ها از پایه و بدون وابستگی به کتابخانه‌های آماده طراحی شده‌اند. در ادامه، روند کار دقیق و تحلیل نماد O بزرگ (Big-O) برای هر یک از الگوریتم‌های پیاده‌سازی شده در سیستم به تفکیک ارائه و اثبات می‌شود.

1 - BFS :

هدف این متد، شروع حرکت از یک گره مبدا و پیمایش گراف به صورت لایه به لایه است. همزمان با این پیمایش، فاصله دقیق (تعداد یال‌ها) از گره مبدا تا تمام گره‌های متصل به آن نیز محاسبه و ذخیره می‌شود.

1. مقداردهی اولیه : ابتدا سه ساختمان داده کلیدی ایجاد می‌شوند:

- یک Queue (صف) برای مدیریت گره‌هایی که باید پردازش شوند.

- یک HashSet برای ثبت گره‌های ملاقات شده (جلوگیری از افتادن در حلقه بی‌نهایت).

- یک Dictionary برای نگهداری فاصله هر گره تا مبدا.

2. گره مبدا وارد صف و مجموعه visitedSet شده و فاصله آن تا خودش برابر 0 تنظیم می‌شود.

3. حلقه اصلی پیمایش : تا زمانی که صف خالی نشده است، گره جلوی صف خارج می‌شود.

4. لیست دوستان (همسایگان) این گره از ساختار گراف دریافت می‌شود.

5. به ازای هر دوست، اگر قبلاً ملاقات نشده باشد، به عنوان یک گره جدید علامت‌گذاری شده، وارد صف می‌شود و فاصله آن دقیقاً برابر با فاصله گره فعلی به علاوه یک ثبت می‌گردد.

6. در نهایت، سیستم شیء BFSResult را که حاوی لیست گره‌های پیمایش شده و دیکشنری فواصل آن‌هاست، برمی‌گرداند.

پیچیدگی زمان (Time Complexity)

در این الگوریتم، هر گره (کاربر) دقیقاً یک بار وارد صف و از آن خارج می‌شود. همچنین در طول اجرای کامل الگوریتم، هر یال (رابطه دوستی) متعلق به گره‌های متصل به مبدا، حداکثر دو بار بررسی می‌شود (چون گراف بدون جهت است). از آنجایی که عملیات بررسی وجود یک گره در HashSet و افزودن به آن دارای زمان اجرای ثابت $O(1)$ است، پیچیدگی زمانی نهایی الگوریتم برابر با $O(V + E)$ خواهد بود که در آن V تعداد کاربران (گره‌ها) و E تعداد روابط دوستی (یال‌ها) در مؤلفه همبند مربوطه است. این بهینه‌ترین زمان ممکن برای پیمایش یک گراف بدون وزن است.

2 - DFS :

1. مقداردهی اولیه : ابتدا یک Stack (پشته) برای مدیریت گره‌ها با منطق LIFO (آخرین ورودی، اولین خروجی)، یک HashSet برای پیگیری گره‌های ملاقات‌شده، و یک List برای ذخیره ترتیب گره‌های پیمایش‌شده ایجاد می‌شود.
2. گره مبدا وارد HashSet (علامت‌گذاری به عنوان ملاقات‌شده) و پشته می‌شود.
3. حلقه اصلی پیمایش : تا زمانی که پشته خالی نشده است، بالاترین گره از پشته خارج (Pop) شده و به لیست نتیجه اضافه می‌شود.
4. لیست دوستان (همسایگان) این گره از گراف دریافت می‌شود.
5. به ازای تک‌تک دوستان، اگر گره مورد نظر در visitedSet وجود نداشته باشد، بلافاصله به عنوان ملاقات‌شده علامت می‌خورد و وارد پشته (Push) می‌شود. به دلیل ماهیت LIFO در پشته، آخرین همسایه‌ای که وارد پشته می‌شود، در دور بعدی حلقه بلافاصله پردازش شده و حرکت در عمق گراف ادامه می‌یابد.

پیچیدگی زمانی (Time Complexity)

استراتژی ثبت گره‌ها در visitedSet دقیقاً پیش از ورود به پشته (Push)، تضمین می‌کند که هیچ گرهی بیش از یک بار وارد پشته نشود. در نتیجه، حلقه بیرونی (while) دقیقاً به تعداد گره‌های قابل دسترس (V) تکرار می‌شود. در حلقه

داخلی، تمام یال‌های متصل به گره فعلی بررسی می‌شوند. از آنجایی که در یک گراف بدون جهت هر یال دقیقاً دو بار (یک بار از سمت هر یک از دو گره متصل به آن) بررسی می‌شود، مجموع بررسی یال‌ها برابر با $2E$ خواهد بود. با توجه به اینکه تمام عملیات‌های درج در HashSet، جستجو در آن و کار با پشته دارای مرتبه زمانی ثابت $O(1)$ هستند، پیچیدگی زمانی کل این الگوریتم به بهینه‌ترین حالت ممکن یعنی $O(V + E)$ می‌رسد.

3 - Adamic Adar :

این الگوریتم یکی از معیارهای هوشمندانه و قدرتمند در تئوری گراف برای پیشنهاد دوست و سنجش میزان ارتباط دو گره است. ایده اصلی این روش بر این منطق استوار است که دوستان مشترکی که خلوت‌تر هستند و دوستان کمتری دارند، نشان‌دهنده پیوند عمیق‌تری بین دو فرد هستند. تشریح روند کار :

این الگوریتم میزان شباهت بین دو گره ورودی nodeA و nodeB را طبق مراحل زیر محاسبه می‌کند:

1. اعتبارسنجی ورودی‌ها: (Validation)

- اگر هر دو گره یکسان باشند ($nodeA == nodeB$)، امتیاز صفر برگردانده می‌شود.
- وجود هر دو گره در شبکه و داشتن حداقل یک دوست برای هر کدام بررسی می‌شود. در صورت عدم برقرار بودن این شرایط، امتیاز صفر بازگردانده می‌شود.

2. استخراج دوستان مشترک: (Finding Common Friends)

- لیست دوستان nodeA و nodeB استخراج می‌شود.
- برای بهینه‌سازی سرعت جستجو، دوستان nodeB وارد یک HashSet با زمان دسترسی $O(1)$ می‌شوند.
- با پیمایش روی دوستان nodeA و شرط بررسی وجود در HashSet مربوط به nodeB، تمامی دوستان مشترک در یک لیست ذخیره می‌شوند.

3. محاسبه امتیاز وزن‌دار: (Scoring)

- به ازای هر دوست مشترک، درجه (تعداد کل دوستان) آن گره محاسبه می‌شود.
- اگر درجه آن دوست مشترک بزرگتر از ۱ باشد، وزن معکوس لگاریتمی آن با فرمول $\frac{1}{\log(\text{degree})}$ محاسبه شده و به totalScore اضافه می‌شود.

4. بازگرداندن نتیجه : مجموع امتیازات محاسبه شده به عنوان وزن شباهت دو گره برگردانده می‌شود.

تحلیل پیچیدگی زمانی (Time Complexity)

فرض کنیم d_A درجه (تعداد دوستان) گره A و d_B درجه گره B باشد:

- دریافت لیست‌ها و تبدیل به HashSet: دریافت لیست همسایگان در ساختار لیست مجاورت زمان $O(1)$ دارد. تبدیل لیست دوستان گره B به HashSet دقیقاً نیازمند $O(d_B)$ عملیات است.
 - پیدا کردن دوستان مشترک: پیمایش روی d_A دوست گره A و چک کردن هر کدام در HashSet (با زمان ثابت $O(1)$)، زمانی برابر با $O(d_A)$ می‌برد.
 - محاسبه امتیاز لگاریتمی: تعداد دوستان مشترک حداکثر برابر با $\min(d_A, d_B)$ است. به ازای هر دوست مشترک، تابع $\text{Count}()$ زمان $O(1)$ برمی‌گرداند و محاسبه لگاریتم نیز زمان ثابت $O(1)$ دارد.
- بنابراین، پیچیدگی زمانی کل الگوریتم برابر با $O(d_A + d_B)$ است. این بدین معناست که اجرای الگوریتم کاملاً مستقل از کل اندازه شبکه بوده و فقط و فقط به تعداد دوستان دو گره مورد نظر وابسته است که باعث سرعت اجرای بالای آن در مقیاس‌های بزرگ می‌شود.

4 - Average Degree :

روند کار : این الگوریتم ابتدا تمام کاربران شبکه را پیمایش کرده، تعداد دوستان (درجه) هر کاربر را حساب و با بقیه جمع می‌کند. در نهایت حاصل جمع کل دوستی‌ها بر تعداد کاربران شبکه تقسیم می‌شود تا میانگین روابط به ازای هر فرد مشخص گردد.

تحلیل پیچیدگی زمانی (Time Complexity)

پیمایش تمامی گره‌های شبکه به $O(V)$ زمان نیاز دارد و گرفتن تعداد دوستان هر گره در زمان ثابت $O(1)$ انجام می‌شود. بنابراین پیچیدگی زمانی کل الگوریتم برابر با $O(V)$ است.

5 - Average Path Length :

این الگوریتم میانگین طول کوتاه‌ترین مسیر بین تمامی جفت کاربران دارای ارتباط در شبکه را محاسبه می‌کند:

1. اعتبارسنجی اولیه : اگر تعداد کل گره‌های شبکه ۱ یا کمتر باشد، میانگین مسیر صفر برگردانده می‌شود.
2. اجرای BFS برای تمامی گره‌ها : به ازای هر کاربر در شبکه (startNode) ، الگوریتم پیمایش سطح اول (BFS) اجرا می‌شود تا فاصله کوتاه‌ترین مسیر از آن کاربر به تمامی گره‌های قابل دسترس مشخص گردد.
3. محاسبه مجموع فواصل : فواصل تمام گره‌های مقصد (به جز خود گره مبدا) در متغیر sumOfAllPaths جمع شده و تعداد جفت‌های مرتبط در reachablePairsCount شمارش می‌شود.
4. محاسبه میانگین : در نهایت با تقسیم مجموع فواصل بر تعداد کل جفت‌های قابل دسترس، میانگین طول مسیر کل شبکه برگردانده می‌شود (در صورت عدم وجود هرگونه مسیر، مقدار صفر برمی‌گردد).

تحلیل پیچیدگی زمانی (Time Complexity)

- اجرای الگوریتم BFS برای یک گره به تنهایی نیازمند $O(V + E)$ زمان است.
- چون الگوریتم BFS به ازای تک‌تک گره‌های شبکه اجرا می‌شود و حلقه جمع‌آوری فواصل نیز به ازای هر گره زمانی حداکثر $O(V)$ می‌برد، پیچیدگی زمانی کل الگوریتم برابر با $O(V \times (V + E))$ یا به عبارتی $O(V^2 + VE)$ خواهد بود.

6 - Betweenness Centrality :

مرکزیت بینابینی شاخصی برای سنجش میزان اهمیت و تسلط یک کاربر بر جریان اطلاعات در شبکه اجتماعی است. کاربری دارای بالاترین مرکزیت بینابینی است که بیشترین تعداد از کوتاهترین مسیرهای بین جفت‌های مختلف کاربران از او عبور کند. این افراد «پل‌های ارتباطی» یا گلوگاه‌های شبکه محسوب می‌شوند و در صورت حذف آن‌ها از شبکه، ارتباط بین سایر بخش‌ها قطع شده یا فواصل افراد به شدت زیاد می‌شود.

روش‌های سنتی برای محاسبه این متریک نیازمند محاسبه جداگانه تمامی جفت‌مسیرها بودند که پیچیدگی زمانی سنگین $O(V^3)$ را تحمیل می‌کردند. الگوریتم اولریک براندز (Ulrik Brandes) با تلفیق پیمایش BFS و برنامه‌نویسی پویا، این زمان را به شدت کاهش داده است.

1. اعتبارسنجی اولیه : اگر تعداد کاربران شبکه ۲ یا کمتر باشد، هیچ پل ارتباطی معنی‌داری وجود ندارد و امتیازها صفر برگردانده می‌شوند.

2. اجرای متد BrandesBFS برای تک‌تک گره‌ها:

○ پاس رو به جلو : (Forward Pass – BFS) پیمایش سطح اول برای محاسبه فواصل، تعداد کوتاهترین مسیرها (pathCounts) و ذخیره والدین هر گره انجام شده و گره‌ها بر اساس ترتیب دیده‌شدن وارد یک Stack می‌شوند.

○ پاس رو به عقب : (Backward Pass – Accumulation) با خالی کردن پشته از آخر به اول، میزان وابستگی گره‌های بالادستی بر اساس نسبت مسیرهای گذرنده محاسبه می‌شود.

3. مجموع امتیازات و نرمال‌سازی : مجموع امتیازهای حاصل از تمام پیمایش‌ها جمع شده و چون گراف بدون جهت است (هر مسیر دو بار شمرده شده)، بر عدد ۲ تقسیم می‌شوند. در نهایت، کاربر یا کاربران دارای بالاترین امتیاز مرکزیت (mostInfluentialNodes) مشخص می‌گردند.

تحلیل پیچیدگی زمانی (Time Complexity)

• روش‌های سنتی محاسبه این متریک نیازمند $O(V^3)$ زمان هستند، اما الگوریتم براندز آن را بسیار بهینه کرده است.

- اجرای الگوریتم BFS و پاس رو به عقب بر روی پشته به ازای یک گره مبدا نیازمند $O(V + E)$ زمان است.
- با تکرار این فرآیند برای تمامی V گره شبکه، پیچیدگی زمانی کل الگوریتم به $O(V \times (V + E))$ یا به عبارتی $O(V^2 + VE)$ می‌رسد.

7 - Closeness Centrality :

این الگوریتم، میزان متوسط فاصله یک کاربر از سایر کاربران را می‌سنجد؛ هرچه مجموع فواصل یک کاربر تا بقیه کمتر باشد، به مرکز شبکه نزدیک‌تر است و سریع‌تر می‌تواند به همه دسترسی داشته باشد:

1. اعتبارسنجی اولیه : اگر تعداد کاربران شبکه ۱ یا کمتر باشد، امتیاز صفر برگردانده می‌شود.
2. محاسبه فواصل با : ClosenessBFS به ازای هر کاربر در شبکه، الگوریتم BFS اجرا می‌شود تا کوتاه‌ترین فاصله او تا تمام گره‌های دیگر به دست آید.
3. محاسبه امتیاز نزدیکی:
 - فواصل تمام اعضا جمع زده می‌شود. (sumOfDistances)
 - در صورت همبند بودن شبکه برای آن گره، امتیاز نزدیکی با تقسیم «تعداد کل کاربران منهای یک» بر «مجموع فواصل» محاسبه می‌شود:

$$\text{Closeness Score} = \frac{V - 1}{\sum \text{Distances}}$$

4. شناسایی گره‌های مرکزی : امتیاز محاسبه‌شده ذخیره شده و گره یا گره‌هایی که بالاترین امتیاز نزدیکی (maxScore) را کسب کنند، در لیست CentralityNodes قرار می‌گیرند.

تحلیل پیچیدگی زمانی (Time Complexity)

- اجرای متد ClosenessBFS و محاسبه مجموع فواصل برای یک گره نیازمند $O(V + E)$ زمان است.

- چون این عملیات به ازای تک تک V گره موجود در شبکه تکرار می شود، پیچیدگی زمانی کل الگوریتم برابر با $O(V \times (V + E))$ یا به عبارتی $O(V^2 + VE)$ خواهد بود

8 - Common Neighbors :

این الگوریتم لیست و تعداد دقیق همسایه ها (دوستان) مشترک بین دو کاربر مشخص را استخراج می کند:

1. اعتبارسنجی اولیه : یکسان بودن دو گره، عدم وجود گره ها در شبکه یا خالی بودن لیست دوستان هر یک از آن ها بررسی شده و در صورت وجود مشکل، عدد صفر بازگردانده می شود.
2. بهینه سازی جستجو : لیست دوستان nodeB استخراج شده و برای دستیابی به زمان جستجوی ثابت $O(1)$ ، درون یک HashSet قرار می گیرد.
3. استخراج اشتراک ها : الگوریتم روی دوستان nodeA پیمایش کرده و با بررسی شرط وجود در HashSet دوستان گره B ، لیست افراد مشترک را تشکیل می دهد.
4. بازگرداندن نتیجه : در نهایت لیست دوستان مشترک (SharedNeighbors) و تعداد آن ها (count) در خروجی تنظیم و ارجاع داده می شود.

تحلیل پیچیدگی زمانی (Time Complexity)

- فرض کنیم d_A برابر با تعداد دوستان گره A درجه A و d_B برابر با تعداد دوستان گره B باشد.
- تبدیل لیست دوستان گره B به HashSet نیازمند $O(d_B)$ زمان است.
- پیمایش لیست دوستان گره A و بررسی وجود هر کدام در مجموعه نیازمند $O(d_A)$ زمان است.
- در نتیجه، پیچیدگی زمانی کل الگوریتم برابر با $O(d_A + d_B)$ است. این زمان کاملاً مستقل از اندازه کل شبکه V و E (بوده و تنها به تعداد دوستان دو کاربر مورد نظر بستگی دارد).

9 - Community Detection :

این الگوریتم شبکه بزرگ را به اکیپ‌ها و جوامعی تفکیک می‌کند که ارتباطات درون‌گروهی متراکم و ارتباطات برون‌گروهی اندکی دارند:

1. مقداردهی اولیه و مرتب‌سازی : ابتدا درجه (تعداد دوستان) تمام کاربران محاسبه شده و کاربران بر اساس درجه‌شان به صورت صعودی مرتب می‌شوند. در ابتدا هر کاربر در یک جامعه مستقل متعلق به خودش قرار می‌گیرد.

2. فاز اول - تفکیک جوامع محلی: (Local Communities)

○ متد RunLouvainPass با شرط غیرسخت ($\geq$) اجرا می‌شود.

○ در هر تکرار (حداکثر ۱۰ دور)، هر کاربر جامعه دوستان مجاورش را بررسی کرده و اگر بیشترین همسایگانش در جامعه دیگری باشند، به آن جامعه منتقل می‌شود.

3. فاز دوم - تثبیت جوامع سراسری: (Global Communities)

○ همان فاز مجدداً اما با شرط سخت‌گیرانه ($>$) اجرا می‌شود تا مرزهای جوامع پایدارتر شده و از نوسانات لحظه‌ای جلوگیری شود.

4. استخراج و دسته‌بندی : در نهایت با تابع ExtractCommunitiesFromMap کاربران بر اساس آیدی جامعه دسته‌بندی شده و لیست گروه‌های محلی و سراسری بازگردانده می‌شود.

تحلیل پیچیدگی زمانی (Time Complexity)

• مرتب‌سازی گره‌ها : مرتب‌سازی V کاربر بر اساس درجه نیازمند $O(V \log V)$ زمان است.

• اجرای الگوریتم لووین: (RunLouvainPass)

○ این تابع حداکثر به تعداد ثابتی ($I \leq 10$) تکرار می‌شود.

○ در هر تکرار، تمامی گره‌ها بررسی شده و مجموع همسایگان پیمایش شده برابر با $2E$ است.

○ بنابراین زمان اجرای هر پاس برابر است با $O(I \times (V + E))$ که با توجه به ثابت بودن I برابر با $O(V + E)$ خواهد بود.

- استخراج گروه‌ها: پیمایش دیکشنری نیازمند $O(V)$ زمان است.
- در نتیجه، پیچیدگی زمانی کل الگوریتم برابر با $O(V \log V + E)$ است

10 - Connected Components :

این الگوریتم زیرگراف‌های متصل و جدا از هم شبکه را شناسایی می‌کند:

1. مقداردهی اولیه: یک لیست برای ذخیره گروه‌ها (components) و یک مجموعه برای ثبت کاربران دیده‌شده (visitedNodes) ایجاد می‌شود.
2. پیمایش سراسری: تمام کاربران شبکه پیمایش می‌شوند. اگر کاربری قبلاً دیده نشده باشد، به عنوان نقطه شروع یک گروه جدید در نظر گرفته می‌شود.
3. اجرای BFS: الگوریتم BFS از آن کاربر شروع شده و تمامی افراد متصل به او را پیدا می‌کند.
4. ثبت گروه: لیست افراد ملاقات‌شده به عنوان یک مؤلفه همبند به لیست کل اضافه شده و به مجموعه visitedNodes ملحق می‌شوند تا در دورهای بعدی دوباره بررسی نشوند. در نهایت لیست تمام گروه‌ها و تعدادشان برگردانده می‌شود.

تحلیل پیچیدگی زمانی (Time Complexity)

- حلقه اصلی تمامی V گره شبکه را بررسی می‌کند، اما به لطف مجموعه visitedNodes، الگوریتم BFS دقیقاً فقط یک بار برای هر مؤلفه همبند (و در مجموع برای تمامی گره‌ها) اجرا می‌شود.
- اجرای مجموع پیمایش‌ها بر روی تمامی گره‌ها و یال‌ها زمان $O(V + E)$ را مصرف می‌کند.
- بنابراین پیچیدگی زمانی کل الگوریتم برابر با $O(V + E)$ است.

11 - Degree Centrality :

این الگوریتم محبوبیت مستقیم هر کاربر را بر اساس تعداد ارتباطاتش محاسبه کرده و کاربرانی را که بیشترین دوست را در کل شبکه دارند مشخص می‌سازد:

1. پیمایش کاربران : الگوریتم تمامی کاربران موجود در شبکه را پیمایش می‌کند.
2. محاسبه درجه (تعداد دوستان) : به ازای هر کاربر، لیست دوستان دریافت شده و تعداد آن‌ها شمارش می‌شود و در دیکشنری DegreeOfNodes ذخیره می‌گردد.
3. شناسایی بیشترین درجه : تعداد دوستان کاربر با متغیر maxDegree مقایسه می‌شود:
 - اگر تعداد دوستان کاربر از maxDegree بیشتر باشد، مقدار بیشترین درجه به‌روزرسانی شده و لیست کاربران برتر بازنشانی می‌شود.
 - اگر برابر با maxDegree باشد، کاربر به لیست محبوب‌ترین‌ها (CentralityNodes) اضافه می‌شود.
4. بازگرداندن نتیجه : در نهایت دیکشنری درجه‌ها و لیست کاربران دارای بیشترین دوست برگردانده می‌شود.

تحلیل پیچیدگی زمانی (Time Complexity)

- پیمایش تمامی V کاربر موجود در شبکه به $O(V)$ زمان نیاز دارد.
- استخراج تعداد همسایگان هر گره از ساختار لیست مجاورت زمان ثابت $O(1)$ می‌برد.
- در نتیجه، پیچیدگی زمانی کل الگوریتم برابر با $O(V)$ است.

12 - Diameter :

این الگوریتم بیشترین فاصله ممکن بین دو فرد در شبکه (طولانی‌ترین کوتاه‌ترین مسیر) را محاسبه می‌کند:

1. اعتبارسنجی اولیه : اگر تعداد کاربران شبکه ۱ یا کمتر باشد، قطر شبکه صفر بازگردانده می‌شود.

2. اجرای سراسری : الگوریتم به ازای هر کاربر در شبکه ، یک بار پیمایش BFS اجرا می کند تا فواصل کوتاه ترین مسیر از آن گره به تمام گره های قابل دسترس به دست آید.

3. یافتن بیشترین فاصله : در هر بار اجرای BFS ، تمام فواصل به دست آمده در دیکشنری پیمایش شده و بیشترین فاصله مشاهده شده در متغیر maxShortestPath نگهداری می شود.

4. بازگرداندن نتیجه : در نهایت بزرگ ترین فاصله به دست آمده از کل پیمایش ها به عنوان قطر شبکه برگردانده می شود.

تحلیل پیچیدگی زمانی (Time Complexity)

- اجرای الگوریتم BFS برای یک گره زمان $O(V + E)$ مصرف می کند.
- از آنجایی که الگوریتم BFS دقیقاً V بار (به ازای تمام کاربران شبکه) تکرار شده و بررسی فواصل نیز زمانی متناسب با تعداد گره ها می برد، پیچیدگی زمانی کل الگوریتم برابر با $O(V \times (V + E))$ یا به عبارتی $O(V^2 + VE)$ خواهد بود.

13 - Distances From All Users :

این الگوریتم کوتاه ترین فاصله بین یک کاربر مشخص و تک تک کاربران موجود در شبکه را مشخص کرده و آن ها را بر اساس میزان نزدیک بودن مرتب می سازد:

1. اجرای BFS : ابتدا الگوریتم پیمایش سطح اول (BFS) از کاربر مبدا اجرا می شود تا کوتاه ترین مسیرها تا تمامی گره های متصل محاسبه گردد.

2. بررسی و مقداردهی تمام گره ها : تمامی کاربران شبکه پیمایش می شوند:

- برای گره مبدا، فاصله 0 ثبت می شود.
- برای گره های قابل دسترس، فاصله واقعی محاسبه شده در BFS قرار داده می شود.

○ برای گره‌هایی که به گره مبدا متصل نیستند (در مؤلفه همبند دیگری قرار دارند)، مقدار بی‌نهایت مثبت (double.PositiveInfinity) تنظیم می‌شود.

3. مرتب‌سازی: دیکشنری فواصل بر اساس مقدار فاصله (از نزدیک‌ترین کاربر تا دورترین / غیرقابل دسترس‌ترین) به صورت صعودی مرتب‌سازی شده و در خروجی برگردانده می‌شود.

تحلیل پیچیدگی زمانی (Time Complexity)

- اجرای BFS: پیمایش لایه‌ای از گره مبدا نیازمند $O(V + E)$ زمان است.
 - پیمایش و تنظیم فواصل: پیمایش تمام گره‌های شبکه و مقداردهی فواصل نیازمند $O(V)$ زمان است.
 - مرتب‌سازی نتایج: مرتب‌سازی دیکشنری حاوی V گره بر اساس فاصله، زمان $O(V \log V)$ مصرف می‌کند.
- در نتیجه، پیچیدگی زمانی کل الگوریتم برابر با $O(V \log V + E)$ است

14 - Jaccard :

این الگوریتم نسبت تعداد دوستان مشترک دو کاربر به کل دوستان منحصر به فرد آن‌ها (اجتماع دوستان) را محاسبه می‌کند:

1. اعتبارسنجی اولیه: یکسان بودن دو گره، عدم وجود کاربران در شبکه یا نداشتن دوست بررسی شده و در صورت عدم برقراری شرایط، امتیاز صفر برگردانده می‌شود.
2. محاسبه اشتراک (دوستان مشترک): لیست دوستان nodeB وارد یک HashSet با زمان دسترسی $O(1)$ شده و با پیمایش روی دوستان nodeA، تعداد دوستان مشترک به دست می‌آید.
3. محاسبه اجتماع و ضریب ژاکارد: تعداد کل دوستان منحصر به فرد دو کاربر از فرمول زیر محاسبه می‌شود:

$$\text{Union} = d_A + d_B - \text{Intersection}$$

سپس با تقسیم تعداد دوستان مشترک بر اجتماع، ضریب شباهت نهایی (عدد بین ۰ و ۱) حاصل می‌شود:

$$\text{Jaccard Score} = \frac{|N(A) \cap N(B)|}{|N(A) \cup N(B)|}$$

تحلیل پیچیدگی زمانی (Time Complexity)

- فرض کنیم d_A تعداد دوستان گره A و d_B تعداد دوستان گره B باشد.
- ساخت HashSet از لیست دوستان گره B زمان $O(d_B)$ می‌برد.
- پیمایش لیست دوستان گره A و بررسی وجود هر کدام در مجموعه زمان $O(d_A)$ مصرف می‌کند.
- در نتیجه، پیچیدگی زمانی کل الگوریتم برابر با $O(d_A + d_B)$ است که کاملاً مستقل از اندازه کل شبکه بوده و فقط به تعداد دوستان دو کاربر مورد نظر بستگی دارد.

15 - Link Prediction :

این الگوریتم با سنجش شباهت ساختاری بین یک کاربر مشخص (userId) و سایر کاربران غیردوست، برترین افراد را برای پیشنهاد دوستی معرفی می‌کند:

1. فیلتر کردن دوستان فعلی : لیست دوستان فعلی کاربر استخراج شده و درون یک HashSet قرار می‌گیرد تا خود کاربر و افرادی که در حال حاضر با او دوست هستند از لیست پیشنهادات کنار گذاشته شوند.
2. محاسبه شباهت با تمامی کاندیداها : الگوریتم تمامی کاربران شبکه را پیمایش می‌کند. به ازای هر کاربر کاندیدا (که دوست فعلی نیست)، سه الگوریتم سنجش شباهت به صورت هم‌زمان فراخوانی می‌شوند:

○ تعداد دوستان مشترک (Common Neighbors)

○ ضریب شباهت ژاکارد (Jaccard)

○ امتیاز وزن دار آدامیک-آدار (Adamic-Adar)

3. استخراج برترین‌ها : (topK) امتیازات مثبت به دست آمده در سه لیست جداگانه ذخیره می‌شوند. هر لیست به صورت نزولی (از بیشترین شباهت به کمترین) مرتب شده و K پیشنهاد برتر هر الگوریتم خروجی داده می‌شود.

تحلیل پیچیدگی زمانی (Time Complexity)

- فرض کنیم V تعداد کل کاربران، E تعداد روابط دوستی، و d_u تعداد دوستان کاربر مورد نظر باشد.
- محاسبه سه معیار شباهت برای یک کاندیدا با درجه d_x نیازمند زمان $O(d_u + d_x)$ است.
- اجرای این ارزیابی روی تمامی V کاندیدای موجود در شبکه، زمانی برابر با مجموع درجات خواهد داشت:

$$\sum_{x \in V} O(d_u + d_x) = O(V \cdot d_u + 2E) = O(V \cdot d_u + E)$$

- مرتب‌سازی لیست‌های حاصل با حداکثر V عضو نیازمند زمان $O(V \log V)$ است.

در نتیجه، پیچیدگی زمانی کل الگوریتم برابر با

$$O(V \cdot d_u + V \log V + E)$$

است، که V تعداد کاربران، E تعداد کل روابط و d_u تعداد دوستان کاربر مورد نظر است.

16 - Network Information :

این الگوریتم آمار کلان و متریک‌های ساختاری کلی گراف را تجمیع کرده و به عنوان یک گزارش جامع ارائه می‌دهد:

1. شمارش پایه و پرمخاطب‌ترین کاربران: تعداد کل کاربران (V) و روابط دوستی (E) دریافت شده و با یک بار پیمایش گره‌ها، کاربر یا کاربرانی که بیشترین دوست را دارند (UsersWithMostFriends) شناسایی می‌شوند.
2. محاسبه شاخص‌های ساختاری:

○ میانگین دوستی به ازای هر کاربر: حاصل ضرب $2 \times E$ تقسیم بر تعداد کاربران.

○ تراکم شبکه (Density): نسبت تعداد یال‌های موجود به حداکثر یال‌های ممکن با فرمول $\frac{2E}{V(V-1)}$.

3. تحلیل زیرگراف‌ها و ابعاد شبکه:

○ اجرای الگوریتم ConnectedComponents و انتخاب بزرگ‌ترین گروه دوستی متصل به هم.

○ اجرای الگوریتم Diameter برای محاسبه قطر کل شبکه (طولانی‌ترین کوتاه‌ترین مسیرهای موجود).

تحلیل پیچیدگی زمانی (Time Complexity)

- محاسبه درجه، تراکم و آمار پایه: نیازمند $O(V)$ زمان است.
 - اجرای مولفه‌های همبند: نیازمند $O(V + E)$ زمان است.
 - محاسبه قطر شبکه (حلقه BFS سراسری): نیازمند $O(V \times (V + E))$ زمان است.
- از آنجایی که الگوریتم محاسبه قطر شبکه بیشترین زمان را مصرف می‌کند، پیچیدگی زمانی کل این کلاس توسط آن تعیین شده و برابر با
- $$O(V^2 + VE)$$
- خواهد بود

17 - Path Exist :

این الگوریتم تعیین می‌کند که آیا بین دو کاربر مشخص هیچ‌گونه مسیر ارتباطی یا زنجیره‌ای از دوستی وجود دارد یا خیر:

1. اجرای BFS : الگوریتم پیمایش سطح اول (BFS) از کاربر مبدا اجرا می‌شود تا تمامی گره‌های قابل دسترس در مؤلفه همبند او پیدا شوند.

2. بررسی حضور گره مقصد : لیست گره‌های ملاقات‌شده پیمایش شده و وجود داشتن کاربر مقصد در آن چک می‌شود.

3. بازگرداندن نتیجه : در صورت پیدا شدن گره مقصد در لیست، مقدار isPathExist برابر با true و در غیر این صورت برابر با false تنظیم شده و برگردانده می‌شود.

تحلیل پیچیدگی زمانی (Time Complexity)

- اجرای BFS : در بدترین حالت (زمانی که شبکه یکپارچه باشد یا کل شبکه پیمایش شود)، پیمایش لایه‌ای به $O(V + E)$ زمان نیاز دارد.

- جستجو در لیست VisitedNodes پیمایش خطی روی لیست گره‌های ملاقات شده حداکثر $O(V)$ زمان می‌برد.

در نتیجه، پیچیدگی زمانی کل الگوریتم برابر با $O(V + E)$ است

18 - Shortest Path :

این الگوریتم کوتاه‌ترین زنجیره ارتباطی (کمترین تعداد واسطه) بین کاربر مبدا و کاربر مقصد را استخراج می‌کند:

1. اعتبارسنجی اولیه : اگر مبدا و مقصد یکسان باشند، مسافتی وجود ندارد و خود گره به عنوان مسیر تک‌عضوی بازگردانده می‌شود.
2. پیمایش BFS و ثبت والدین (ShortestPathBFS) : پیمایش سطح اول از گره مبدا شروع می‌شود. به ازای هر گره جدید، والد آن در parentMapDictionary ذخیره شده تا مسیر قابل ردگیری باشد. به محض رسیدن به گره مقصد، پیمایش جهت بهینه‌سازی فوراً متوقف می‌شود.
3. بازسازی معکوس مسیر : در صورت دسترسی به مقصد، با شروع از گره مقصد و دنبال کردن زنجیره والدها تا رسیدن به گره مبدا، مسیر استخراج می‌شود.
4. معکوس‌سازی و بازگرداندن نتیجه : آرایه مسیر معکوس می‌شود تا ترتیب صحیح از مبدا به مقصد حاصل شود و خروجی نهایی تنظیم گردد.

تحلیل پیچیدگی زمانی (Time Complexity)

- اجرای BFS : در بدترین حالت (عدم وجود مسیر یا حضور مقصد در انتهای پیمایش)، تمام V گره و E یال شبکه بررسی می‌شوند که زمان $O(V + E)$ می‌برد.
- بازسازی و معکوس‌سازی مسیر : این عملیات متناسب با طول مسیر (L) که حداکثر برابر با V است) زمان $O(L)$ مصرف می‌کند.

در نتیجه، پیچیدگی زمانی کل الگوریتم برابر با $O(V + E)$ است

19 - User Friends List :

این الگوریتم لیست کامل دوستان مستقیم (همسایگان لایه اول) یک کاربر خاص را از گراف شبکه استخراج می کند:

1. فراخوانی همسایگان : با استفاده از متد `graph.GetFriends(user)`، دوستان مستقیم کاربر مورد نظر دریافت می شوند.

2. تبدیل و بسته بندی : خروجی به دست آمده به یک لیست تبدیل شده و در ویژگی `listOfFriends` از شیء `UserFriendsListResult` ذخیره می شود.

3. بازگرداندن نتیجه : شیء نتیجه حاوی لیست دوستان کاربر برگردانده می شود.

تحلیل پیچیدگی زمانی (Time Complexity)

- فرض کنیم d_u برابر با درجه کاربر (تعداد دوستان او) باشد.
- استخراج همسایگان از ساختار لیست مجاورت زمان $O(1)$ می برد و تبدیل آن به یک لیست جدید با `ToList()` زمان $O(d_u)$ مصرف می کند.
- در نتیجه، پیچیدگی زمانی کل الگوریتم برابر با $O(d_u)$ است که کاملاً به تعداد دوستان همان کاربر بستگی دارد.